Eevee

Conversor Campaign Manager (CM) → ALFRED

Rama: `feature/cm-to-alfred`

1. Resumen

Eevee es una herramienta web que convierte el CSV exportado de Campaign Manager (CM) en un Excel (.xlsx) con el formato de IR que ALFRED espera. Es una solución temporal y de bajo mantenimiento: no tiene backend, no usa base de datos y los datos del usuario **nunca salen de su navegador**.

La lógica de negocio está en Python y corre dentro del navegador mediante **Pyodide** (Python compilado a WebAssembly). La interfaz es HTML + JavaScript simple.

2. Arquitectura

```
Usuario (navegador)
  |  1. Sube CSV de CM
  v
index.html + assets/app.js        (UI, lectura de archivo, descarga)
  |  2. Carga Pyodide + openpyxl (CDN jsdelivr, Pyodide v0.27.5)
  |  3. Descarga config/mapping.py y scripts/transform.py y los monta en el FS virtual
  v
scripts/transform.py  --run(csv_text, filename)-->  { excel_bytes, errors, row_count, has_errors
}
  |  usa reglas de config/mapping.py
  v
Excel ALFRED (.xlsx) descargable + vista previa en pantalla
```

Componente	Archivo	Responsabilidad
Interfaz	`index.html`	Página única: zona de carga, vista previa, botón de descarga.
Controlador UI	`assets/app.js`	Inicializa Pyodide, lee el archivo, invoca Python, construye la vista previa y dispara la descarga.
Servidor local	`assets/server.ps1`	Servidor estático de PowerShell para pruebas locales (el sitio no puede abrirse con `file://` por el `fetch` de los módulos).
Motor	`scripts/transform.py`	Parseo del CSV, transformación, validación y generación del Excel.
Reglas de negocio	`config/mapping.py`	Única fuente de verdad: mapeos, valores fijos, catálogos, columnas y obligatorios.
Pruebas	`tests/` , `uat/`	Suite pytest y libro UAT para Key Users.

> **Principio de diseño:** las reglas de negocio viven en `config/mapping.py` ; el código en `transform.py` solo las aplica. Un cambio de regla (nuevo país, nueva opción de dropdown, nuevo campo obligatorio) normalmente se hace sin tocar el motor.

3. Flujo de ejecución en el navegador

1. **Arranque** (`initPyodide`): carga Pyodide, instala `micropip` y `openpyxl`, descarga `mapping.py` y `transform.py` (con parámetro de versión para evitar caché) y los escribe en el sistema de archivos virtual (`/config`, `/scripts`).
2. **Carga del archivo** (`handleFile`): solo acepta `.csv`; lee el contenido como texto.
3. **Procesamiento** (`processCSV`): ejecuta `run(csv_text, filename)` en Python.
4. **Resultado**: guarda los bytes del Excel, la lista de errores y el conteo de filas; construye la vista previa (`buildPreview`) y muestra el botón de descarga.
5. **Descarga** (`triggerDownload`): entrega el .xlsx generado.

4. Pipeline de transformación (`transform.py`)

El punto de entrada es `run(csv_text, filename)`: `parse_csv` → `transform` → `write_excel`. Cualquier excepción se captura y se devuelve como `Fatal error` sin romper la UI.

4.1 Lectura del CSV (`parse_csv`)

- Descarta filas totalmente vacías.
- Detecta la fila de encabezado por mayor coincidencia con las columnas esperadas de CM (tolera una fila decorativa encima del encabezado).

- Rellena filas cortas con vacío.

4.2 Transformación (`transform`) en tres pasadas

Pasada	Qué hace
1. Nivel y herencia	Por cada fila: hereda valores vacíos de las filas ancestro y normaliza la plataforma; determina el nivel (LEVEL).
2. Jerarquía e IDs	`inject_implicit_campaigns` completa niveles faltantes de DV360 (Campaign / IO / Line Item / Ad Group) y `generate_ids` crea los IDs jerárquicos.
3. Armado de filas	Aplica mapeos directos, valores fijos, fechas, montos, país, sub-sector, taxonomía, género/edad, stage, tipo y campos manuales; luego `reconcile_parent_budgets` ajusta presupuestos de padres.

4.3 Reglas de transformación principales

Campo ALFRED	Origen / regla
BRAND, SUB_BRAND, INITIATIVE, PLATFORM, AUTOBIDDERS, BONO, PRICING_MODEL	Copia directa desde CM (`DIRECT_MAPPING`). AUTOBIDDERS se normaliza a Yes/No. PRICING_MODEL se resuelve desde `Cost Type` (con respaldo si viene N/A).
BUY_TYPE, IRA_VERSION, REGION, AUDIENCE, BROADCAST, PRICE_TYPE	Valores fijos (`Auction` , `V1` , `National` , `Demo` , `NA` , `Dynamic`).
START_/END_ DAY, MONTH, YEAR	`parse_date` : acepta `YYYY-MM-DD` , `DD/MM/YYYY` , `MM/DD/YYYY` . Se separa en día/mes/año sin cambiar el valor.
MONTH	Código de mes de la fecha de inicio (JAN, FEB, ... DEC).
BUDGET, TOTAL_BUDGET	`clean_currency` : quita `$` y detecta formato americano (`1,234.56`) o europeo (`1.234,56`) según el último separador.
COUNTRY	Del nombre del archivo: se separa en tokens alfanuméricos y se acepta solo si hay exactamente un código válido (25 países). Si falta, es ambiguo o inválido queda vacío y se reporta; nunca se adivina.
SUB_SECTOR	Texto terminado en "Care" dentro del Line Item.
GENDER / AGE_RANGE	Del `Target Audience` (prefijo W/M/H/P/HM y segmento de edad).
STAGE	De `Media Objective` (Awareness, Consideration, Conversion).
LEVEL, CAMPAIGN_TAXO, ID	Del Line Item: segmentos separados por `->` ; desde el primer segmento con forma de código (`CMP_` , `IO#` , `LI#` , `ADG#`) comienza la jerarquía.
TYPE	`infer_type` según plataforma + nivel (`TYPE_OPTIONS`) y contexto del registro; si no hay certeza queda vacío.
Campos manuales	CAMPAIGN_GOAL, PO, FREQUENCY, STATUS, IMP_STATUS, BID_TYPE_OPTIMIZATION_TYPE, BID_VALUE, KPI_VALUE, PACING, BUDGET_TYPE: se dejan para completar en Excel con dropdowns.
Campos vacíos permitidos	Lista `EMPTY_FIELDS` (21 campos que ALFRED acepta vacíos).

4.4 Plataformas y niveles

Plataforma	Prefijo ID	Niveles	Cómo se determina el nivel
DV360	DV	Campaign, Insertion Order, Line Item, Ad Group	Último código del Line Item: ADG→Ad Group, LI→Line Item, IO→Insertion Order, otro→Campaign.
Meta	M	Campaign, Ad Group	Dos niveles: ADG→Ad Group, cualquier otro código→Campaign.
TikTok	TK	Campaign, Ad Group	
Pinterest	P	Campaign, Ad Group	
Google Ads	S	Campaign, Ad Group	
Amazon Prime	A	Campaign, Ad Group	

El reconocimiento de plataforma (`normalize_platform`) prueba, en orden: `Platform`, alias (Facebook/Instagram/Messenger → Meta), `Publisher` y, como respaldo, `OU Planning Level`. Solo estas 6 plataformas están soportadas; cualquier otra se omite con aviso.

4.5 Generación de IDs

Los IDs son jerárquicos: la raíz usa el prefijo de la plataforma más un contador (`DV1`, `M2`) y cada hijo agrega un sufijo numérico al ID del padre (`DV1-1`, `DV1-1-2`). El contador se lleva por plataforma y por padre.

4.6 Jerarquía implícita y presupuestos

- Si en DV360 faltan niveles intermedios (por ejemplo una Campaign sin su fila), el motor inyecta filas implícitas para que el árbol quede completo. Las filas generadas no copian la taxonomía y los Ad Groups generados no se validan.
- Una Campaign generada inicia en 0 y toma la suma de sus hijos directos. Una Campaign real con presupuesto 0 o vacío también toma la suma de sus hijos; si ya tiene valor, se respeta.

5. Validación

Tras armar las filas, cada una se valida contra `ALFRED_MANDATORY` (28 campos). Reglas condicionales:

- `PACING`, `BID_VALUE`, `KPI_VALUE`: solo DV360 (no Campaign). `BID_VALUE` también TikTok.
- `BUDGET_TYPE`: solo Meta y TikTok.
- Los campos manuales no bloquean ni generan error: se completan en Excel.

Tipo de aviso	Formato	Origen
Fila omitida: plataforma no soportada	`CM row N: Platform 'X' is not supported by Eevee; the row was not included`	Pasada 1
Fila omitida: Line Item sin código	`CM row N: Line Item 'X' has no level code (CMP/ADG/IO/LI); the row was not included`	Pasada 1
Campo obligatorio vacío	`Row N: Missing mandatory field 'X'`	Validación final
Error fatal	`Fatal error: ...`	`run()`

Nota: en "CM row N", N es la posición entre las filas de datos no vacías del CSV de CM. En "Row N", N es la fila del Excel de salida. Una fila sin Line Item ni plataforma (totales o decorativas) se omite sin aviso.

Decisión de producto vigente: **se permite descargar el Excel aunque existan avisos o errores**; estos quedan registrados en la hoja `Errores` del archivo. Un COUNTRY vacío no bloquea la descarga.

6. Salida: Excel (`write_excel`)

- Hoja **ALFRED**: 61 columnas en el orden exacto de `ALFRED_COLUMN_ORDER`; encabezados por sección con colores (negro, morado, azul) y filas de Campaign coloreadas por plataforma.
- **Dropdowns** (validaciones de datos): STATUS, PACING, BUDGET_TYPE, IMP_STATUS, STAGE, CAMPAIGN_GOAL (depende del STAGE), PRICING_MODEL, TYPE (depende de plataforma/nivel),

BID_TYPE (depende del contexto de la fila) y AUTOBIDDERS. Las listas largas se guardan en la hoja oculta `_lists`.

- Hoja **Errores**: lista de avisos de la ejecución.
- Los montos se escriben como números y las fechas separadas en día/mes/año.

7. Cómo modificar reglas (guía de mantenimiento)

Necesidad	Dónde cambiar
Agregar o quitar un país válido	`COUNTRY_CODES` en `mapping.py`
Cambiar un valor fijo	`FIXED_VALUES`
Nueva opción de dropdown (TYPE, PACING, BID_TYPE...)	`TYPE_OPTIONS`, `PACING_OPTIONS`, `BID_TYPE_OPTIONS`, etc.
Cambiar mapeo directo CM → ALFRED	`DIRECT_MAPPING`
Cambiar campos obligatorios	`ALFRED_MANDATORY` y la sección de validación en `transform()`
Cambiar el orden o las columnas de salida	`ALFRED_COLUMN_ORDER`
Nueva plataforma o alias	`PLATFORM_CONFIG`, `PLATFORM_ALIASES`, `TYPE_OPTIONS` y color de Campaign en `write_excel`
Mapeo de objetivo a stage	`STAGE_MAPPING`, `CAMPAIGN_GOAL_BY_STAGE`

Después de cualquier cambio: correr `pytest` y, si el cambio afecta la experiencia del usuario, revisar el libro UAT.

8. Calidad y pruebas

Capa	Ubicación	Contenido
Unitarias	`tests/test_unit_normalizacion.py`, `tests/test_unit_jerarquia.py`	Fechas, montos, países (25 códigos en mayúscula y minúscula, límites y ambigüedad), género/edad, stage, plataforma, jerarquía e IDs.
Integración / flujo	`tests/test_flujo.py`, `tests/test_excel.py`	Flujo completo CSV→Excel, obligatorios, avisos de filas omitidas, estructura y dropdowns del Excel.
Brechas conocidas	`tests/test_brechas_conocidas.py`	Comportamientos deseados aún no implementados, marcados `xfail` estricto (si se corrigen, la prueba avisa).
Fixtures	`tests/fixtures/`	CSV de ejemplo (DV360, multiplataforma, etc.).
UAT	`uat/Eevee_UAT.xlsx`	32 escenarios de negocio para Key Users (Planner, Trader, Digital Expert). Se genera con `uat/build_uat.py`.

Estado actual: **211 pruebas pasan y 9 son brechas conocidas (xfail)**. Se ejecuta con `python -m pytest` desde la raíz y automáticamente en GitHub Actions (`.github/workflows/tests.yml`). Dependencias de desarrollo en `requirements-dev.txt`.

9. Limitaciones y brechas conocidas

Tema	Comportamiento actual
Numeración de filas	Los errores de obligatorios usan la fila del Excel de salida, no la del CSV original.
Idioma de mensajes	Los avisos del motor están en inglés.
Lista BID_TYPE	Una lista de validación en Excel supera 255 caracteres cuando se escribe en línea (límite de Excel).
Plataformas	Solo 6 soportadas (en la práctica se usan 3); el alcance no se espera que crezca.
Lista de errores en la UI	El despliegue de la lista de errores en pantalla está desactivado temporalmente en `app.js`; los avisos se ven en la hoja `Errores` del Excel.
Dependencia de CDN	Requiere acceso a `cdn.jsdelivr.net` para cargar Pyodide y openpyxl.